

W1 分析

O:fsc.g vWn2n;O2=Θ=both;o/w严格;极限判别:lim f/g=O/c∞=o/O/w).1<loglog<log<v<n<n<n log n<n²<n²<n<n;log底无关;∞exp.

log公设:log(xy)=logx+logy · log(x/y)=logx−logy · log n=n · logx = log y · a^f(log a, n)=n · 2^f(log₂,n)=n · a^f(log b, n)=n^f(log b, a)(指数 ∼ 底互换,比怪函数神器) · 换底log_a b=b·ln/ln a(底无关差常数,0里随便换) · **Stirling**:log(n!) =n log n−n·log e+O(log n)=n log n−1.44n=Θ(n log n) · log⁡k·n=o(n⁡k⁡)∀ε>0

0(任意系数的log<任意正幂). Σ:Σi=n(n+1)/2-Σi²=n(n+1)(2n+1)/6=O(n³). 2²=2²(k+1)-1=2×末项. 等比Σr=(rⁿ(n+1)-1)/(r-1).r>1顶端主导,r<1=O(1).H-n=Σ1/i=O(log n) · log(n)=Θ(n log n). **tightest O=最小成立的上界**.

规则:常数戏+并排加取最大;嵌套乘 ★前缀和预处理:B[i]=A[0]+...+A[i](一遍O(n));任意区间和A[i,j]=B[j]-B[i-1] O(1)——枚举所有区间和从O(n²)降O(n²). ★怪函数比增长快:取log再比·log(nⁿ(1.5))=1.5log n·log x(n)=log n; log(2ⁿ/(log n))=vlog n; log(nⁿ(log n))=log⁡n²=2⁡v(log n)<n log log⁡n<n⁡k⁡(1.5)<n⁡k⁡(log log n).

循环陷阱:①内层依赖外层→逐项求和 ②递归循环 (T1P9):for内嵌套while j--,但每轮j+1→j,总共加sn, 减sn→实际O(n)(粗略看O(n²)也对但不紧;记账:每次-1都有先+1买单) ③双doubling嵌套:外i+=2(log n倍),内while j<i: j+=2(log i次)→Σ_{i<[klog n]}k=Θ(log⁡n²) ④外+++内+=起doubling:内层log(n/2),Σlog(n/2)=log n·log n=log(n!),设 Stirling log(n!)=n log n−1.44n ⇒=1.44n=Θ(n)(大多数附近,内层转不了几圈; 别管n,内层转不了几圈; 考察实例(降序)使后n/2轮各≥n/2→Q(n²).

W2 表·栈·队列

ADT=只足行为不实现.

操作	Array	SLL	DLL
按下单头插删	O(1)	O(n)	O(n)
头插删	O(n)	O(1)	O(1)
尾插	O(1)	O(1)	tail O(1)
尾删	O(1)	O(n)	前驱 O(1)
定位插删	O(n)哪	O(n)前驱	O(1)
按值找	O(n)	O(n)	O(n)

SLL插(p后插):①x.next=p.next ②p.next=x(先连新后改旧,反了丢后半):头插x.next=head,head=x. **SLL删**:只需删p的后继.p.next=p.next.next——必须循着前驱,只给x本身 · O(n)找前驱.

DLL插(p后插)4步:①x.prev=p ②x.next=p.next ③p.next.prev=x ④p.next=x. **DLL删x4步**:①x.prev.next=x.next ②x.next.prev=x.prev(两侧绕过x;header/trailer免判空). ③决:插=新节点先指人,再让人指它;删=左右手拉手绕过它.

Stack数组实现:t=顶部索引,size=+1,全O(1);满栈push→翻倍:amortized O(1)(n次push总代价≤3n:每元素付自己+搬运费);▲单次最坏O(1)则n².“amortized worst-case”是MCQ点;若每次只扩+1则总Θ(n²). 用途:undo/调用栈/括号(开压回弹)/DFS/RPN.

Queue链表数组:enq:Q[enjd]=e,size++,end=(start+size)mod N;deq:start=(start+1)mod N;绕回(end<start)是合法形态;用途:BFS/缓冲. Positional 列表置不失效; Iterator:snapshot快照照 vs dynamic动态. ▲核!队列交错出入题:必须逐步模拟枚举每步.

套路★:O(1)均值队列=攒设sum字段同步±,getAverage=sum/size

SLL只读逆序遍历(T2P4)时空权衡(▲DLL沿prev直走=O(n)时O(1)空,无此问题):

额外空间	时间
O(n) 递归栈/整栈	O(n)
O(n) v·n游标+v·n小段(段内压栈弹出)	O(n)
O(vn) 只有游标(段内反复扫)	Θ(n²(3/2))
O(n) cursor反复从头扫	O(n²)

· 单调栈:“下一个更大/可见灯塔”类O(n) · 递归→迭代·显式栈压(状态,args)元组.

Deque:两端进出全O(1)(DLL). 反向打印链表:递归/栈 O(n)空间;原地反转三指针:prev=null,cur=head;循环 {next=cur.next; cur=next=prev; prev=cur; cur=next}最后 head=prev,O(n)时O(1)额外(再走一遍遍历;只读版见下). 摊还:倍增=amortized O(1)(Σ摊还=2n);

+常数=O(n). 两栈=队列三板:①朴素(T2P5)deq=倒 S1→S2弹→倒回,每次O(1)无摊还 ②懒人:弹完不倒回,S2空才倒一次=enq O(1)deq摊还O(1)(每元素一生最多4次,单次worst=O(n) ③预付:enq时倒压·倒回,时刻保持S2最老在顶=enq worst O(n),deq worst O(1); 题面amortized = ②,worst-case deq = ③.

W3 树

定义:唯一root无父,其余恰1父,沿父指针达root(无环).n节点= n-1条边,siblings同父;ordered树兄弟有序. **depth=祖先数**(不含自身,根0);height按边数=最长根→叶;叶高0;△空子树height=-1(平衡因子题);quiz要考 depth与height的off-by-one. proper=0或2孩(ext=int+1); perfect n=2ⁿ(h+1)-1,complete左→右填,h≥log₂(n+1) -1;第层≤2.

遍历O(n):pre=根定(目录),post=根后(磁盘/表达式)=in=左根右,△仅二叉树有定义(BST+降序).level=BFS. Euler:每点恰过3次,在pre下in后post. 免递归状态机(u,来向),O(1)空间:|parent来←记左往右(无左往左);从右左回→记in,进右来(无视作右返);从右左回→记post,爬回parent;

每边走两遍→O(n),靠parent指针不用栈.

preorder(v): visit(v); for each child w: preorder(w) **postorder(v)**: for each child w: postorder(w) visit(v) **inorder(v)** 二叉: if v.left≠null: inorder(v.left); visit(v); if v.right≠null: inorder(v.right) **EulerTour(v)**: visit左侧(←pre); if v.left: eulerTour(v.left) ↓ visit下方(←in); if v.right: eulerTour(v.right); visit右侧(←post)

depth(v): if v=root: 0 else 1+depth(parent(v)) — O(depth) · **height(v)**: if leaf: 0 else 1+max(height(w) for w in children) — O(n).

W4 树(续)

post自底向上: size=L+r+1, height=max+1(叶0空-1) · sum:pre自顶向下: depth=父+1. **depth(v)**:沿parent上爬数步到O(depth);LCA(u,v):深的先上移到同深,再一起逐上移到相遇,O(h). 直径:post返height,max更新h+Lr+2.

递归复杂度口决:每点O(1)且递归全部孩子⇒O(n)(记账:helper每费O(1)+孩),Σ孩=n-1);只递归单孩/父亲⇒O(h). ▲Quiz陷阱:递归parents=O(depth),但!从root调用” — depth=0→O(1)(读清起点);两孩+每点费O(子树!) — 链树T=O(n²);n次顺序遍历空BST→高O(n).

W4 BST+AVL

操作	BST	AVL
search/min/max	O(h),坏O(n)	O(log n)
range[ℓ,k]2	O(h+n)	O(log n+s)
select/rank+(size)	O(h)	O(log n)
min/max/succ	O(h)	O(n)
in-order全遍历	O(n)	O(n)

insert:search失败空位,必是叶. **delete**:叶摘;1孩splice;2孩in-order succ(右子树最左,无左孩)拷key删它. 有序插入退化链.

Range search:u<k1只右;u>k2只左;否则报+两边. **AVL**:|hL-hR|≤1;高度界证明:n(h)=高h[左节点数=1+n(h-1)+n(h-2)≥2n(h-2)-n2²/|h/2|. h≤2log₂n=O(log n)(紧界1.44log n). **trinode restructure**:不平衡点z+更高孩y+更高孙x,按中序排成a<

b<c = b当根,a,c做其孩子;直线LL/RR=单旋,之字L/R/L=双旋;ins后≤1旋,del(含删副):先BST三case删(2孩→拷in-order succ的key,物理删的永远是下面那个小节点),再从删除处沿路回溯逐层更新更高度,失衡就restructure,最坏O(log n)次旋.

增广select(k)第k小:每点存size=L+R+1;在u处设L=左子树size,u在L;子树内找l=L+1(中序|左[节点O(j)]);k<t→返回u;k<t→进左仍找第k;k>t(←L<k-1)→进右找第k-L-L-1(跳过左子树找L+1(中序)→一条路径O(h). rank(x)=多少<x;带计数器下传,每次右拐+L+1,左拐不加.

▲MCQ要把握k>L-1下移项看成L<k-1.

同型扩展★:区间计数=rank(k)-rank<(k1) · 第k大=第n-k+1小(或对称用右点) · 动态中位数=select([n/2]) · 百分比 · 区间求和:改存sum字段同法前缀维护(区间 min/max=range search取边界 · [min,x]完整(无缺) ⇔rank(x)-x=min+1(mock P4) · 能增广的字段必须由两孩O(1)合成(size/sum/min/max/height+),插删沿路O(1)更新不破坏O(h). 选型:一次性找第k小→quickselect O(n);

流式只要前k · 题面+堆O(n log k);动态插删+反复查→增广BST O(n²) · size=width“supports insert/delete and query in O(log n)”就是它).

W5 堆·排序

PQ op	无序表	有序表	堆
insert	O(1)	O(n)	O(log n)
min	O(n)	O(1)	O(1)
extract	O(n)	O(1)	O(log n)
build	—	—	O(n)自底向上

PQ ADT:insert(k,v)/min()/removeMin()/size(min)版max版只能选-1,△不支持sort(按键取). **PQ-sort**=n次insert+n×removeMin:无序表=selection sort O(n²)任意输入;有序表=insertion sort最坏O(n²)=降序输入,精确O(n¹). 循环不变量直接表:selection=「A[0]已排好且≤A[1..n]全部」;

insert=「A[0..i]内部已排好」. 堆=完全树+父<子(左右无序);complete=第0..h-1层全满(2ⁱ),末层最左,last node=最深层最右;2ⁿh≤sum≤2ⁿ(h+1)-1=高+log₂n. 存数组(下标从1起):节点的父亲在i/2,孩子在2i和2i+1;最后一个节点=下标n,新元素插在n+1; 二进制=路径:左孩2=末尾第0,右孩2i+1=添1,父(i/2)=去末孩,根走到节点i;写成二进制去掉最高位,逐位0往左,1往右(i=6=110→10→右、左);所在层=|log₂i|=二进制位数-1.

排序	worst	其他	稳	in-pl
selection	Θ(n²)	best n²	✓	✓
insertion	O(n)	best O(n)=O(n+1)	✓	✓
merge	O(n log n)	恒	✓	✓
heap	O(n log n)	build O(n)	✓	✓
quick	Θ(n²)已序	avg n log n	×	✓

insert:放到last+1(保持完全)再up-heap上滤; **removeMin**:取根,把数到最后元素搬到根,size-1,再down-heap下滤(与较小孩换到归位);都只删一条根→叶路经=O(log n),数组其余未动(唯只维护松的父子序,不是全序). **decrease-key**(Dijkstra原发明):改小堆中某元素的key:每顶点存指回堆下标location(指针(交换时同步换)→O(1)定位+keyget+上滤=O(log n);

Fib堆O(1)摊还:偷懒法:重复插新条目,弹出时丢丢丢,同伙. **heap-sort**原地:build-heap O(n)后反复swap(root, last)+缩堆+下滤=O(n log n). 比较下界Q(n log n)(决策树). 应用:κ路合并O(mk log k)(堆只装k个,每op O(log k);第k小O(n log k);size-k堆,A[i]>堆max→“可安全忽略,不可能是第k小”. **T5套路**:构造worst实例(降序→insertion Q(n²));telescoping:W=Σ(l-i+1);

批判给定算法+找反例(试单元全负/已序).

W6 哈希

方案	get	put	remove
chaining	期望O(1+α)	期望O(1+α)	期望O(1+α)
lin.probing	期望O(1)	期望O(1)	期望O(1)
cuckoo	worst O(1)	期望O(1),可rehash	worst O(1)

Map ADT:key唯一→get/remove无key→null;put(k,v):新键插入返回null,已有键替换value并返回旧值——△返回键=MCQ陷阱(map含2,C,put(2,E)→返回C,值变E). 无序链表实现:put O(1)(若需重O(n)),get/remove O(n). 记号:K=可能键域,m=|K|≥N=N-表长,n=实存键数<m,α=n/N;A[h(x)]存指针→(x,value),lookup table两致命伤:空间O(N)浪费+键必须是小小数.

h=压缩(k mod N,N取质数,前提均匀随机时两键碰撞概率1/N;随机线性MAD ((ak+b)mod p)mod N,p>m质数,a,b∈[1,p-1],a≠0,对任意两键碰撞=1/N)=α·code(顺序敏感必用polynomial x_i,a^d-1)+...+x_d·d.同字符不同排列不再保证碰撞:**Homer**:从最高位起每步x+下位,O(d);

W6 哈希(续)

a级验选,mate/meat/team/ame求和码全碰撞). ▲MCQ别混:“两键碰撞概率=1/N”vs “新键与n个已存键的期望碰撞数≤nN”(线性期望);Markov:α=½ · P[|新键有碰撞|≤½. **universal**族:随机取N(H,V);Pr[h(n)=h(n)]≤1/N.

鸽巢 · 环集(h:环)是固定函数,把m>N>n个可能的键映进n个槽,必某槽撞≥m/n|n|个键映中(对10¹⁰个可能键压10⁴槽→某槽≥10⁴个键);“坏集”=例子专挑这≥10⁴个槽撞当输入→全部碰撞,表退化链O(n). 结论:任何固定h都在坏数据集,哈希只保证对典型/随机数据快,勿自创哈希(h=k mod 2→两方案数Θ(n)).

chaining:每格挂链表,a=平均链长(α=1仍可O(1+n)); ▲put也要先扫链表查重 · put坏环O(n). **probing**:撞→试(i+1)mod M,uf.primary clustering挤团,▲要求α<1;get存名字:找到碰到空格/探满N格(转一整圈) · ▲查“不存在”的关键=一路跑到首个空格(含)才能下“无”结论(h(3)=3→探3,4,5,6). **DEPUNCT**墓碑(remove置墓碑(直接清空空切断探测链);

get沿链继续;put沿迹往首个墓碑/空位,确认无键重后插. 期望探测1/(1-α)次→α有界则O(1);超阈值(Java>0.75/Py>½)→2×新表rehash.

Cuckoo:两表两槽数,k只可能在T₁[h₁(k)]或T₂[h₂(k)]→get/remove至少探2格=worst O(1)(卖点:“最坏情况”也答O(1));put:按表a→驱逐占用者去它另一表备选位,链式;驱逐成环→更大表全rehash;put期望O(1)(条件N>2n,n次put总期望O(n)=均摊O(1)). 判环:驱逐链长>4n即环,或沿途flag. cuckoo图(槽=点,key=边)已有环→可能死循环,树→必停.

铁律:没有方案三操作同时worst O(1) · 题目要worst-case期时禁用hashing(期望界依赖均匀随机假设). 套路★:开2n表保α≤½(设计题第9-1) · 插入序遍历=表+DLL条目互指针全O(1) · 频次统计:value存count,边插边维护运行max·域值有界 · O(1):生日键在[1,365]表730,≥365时前366次必重复 · 取舍: chaining 简单删副费; probing省指针缓存友但怕墓碑堆积;cuckoo!worst O(1)但put可能整表rehash.

Map实现对比:无序表get/put全O(n) · 哈希exp O(1) · 有序数域get O(log n=2)/put O(n)挪 · AVL全O(log n). **Sorted Map 专属(AVL全O(log n))**:firstEntry/lastEntry; ceiling(k)=最小sk/floor(k)=最大sk/lower(k)=最大<sk/higher(k)=最小>sk — ▲带等号vs不带;del:subMap(k₁,k₂)含k₁不含k₂;哈希全做不了(无序). Set=只有key无value的Map;Multiset=key可重复的Set,实现=Map(key,count)(加/删=counts=1);Multiset remove只删1次出现,Set运算addAll=retainAll+removeAll=差S-T(经由Map实现); **Multimap**=key挂value链表,get O(1+s). 实现谱系:无序表O(1)→lookup table(键=小整数)O(1)费空间→hash期望O(1)→AVL有序O(log n)可.

W7 图·BFS/DFS

操作	边表	邻接表	矩阵
查询	O(n+m)	O(n+m)	O(n²)
空边(u,v)	O(m)	O(deg u)	O(1)
incident/Edges	O(m)	O(deg u)	O(1)
加边	O(1)	O(1)	O(n)
BFS/DFS	—	O(n+m)	O(n²)

术语:simple m≤n(n-1)/2=complete;树=连通+m=n-1→连通无环;Σdeg=2m;Δ=最大度,有向Σin=Σout=m.

subgraph S=(U,F):U=顶点,F⊆E,induced G[U]:E[U]→两端都在U的边;component= maximal连通子图. **BFS**过程:①s标记,入队,L={s} ②反复出队u,扫u的每个邻居v未访问·标记记,记v∈L(←1),入队 ③队空止. 性质:L=L∪s恰跑的边(=无权最短路);每条非树边连同层或相邻层(层不可跨两层).

DFS递归:DFS-visit(u):①进门先标记u ②扫u邻接表,每条边(u,v):未访问→记tree edge,parent[w]=u,立刻递归进w(不扫完其他邻居=往深走);w已访问且w≠parent(u)→back edge报环 ③for未访问自动return=回溯;递归树=当前路径. 主入口对每个未访问点一次(盖到所有分量);每点进一次+走向两端各扫一次 · O(n+m). 边只两类: **tree edge**(走向新点)/**back edge**(指回线上祖先);有back edge⇒有环. **BFS边({u,parent(u)})构成生成树**. 删剖/删剖(level+down and up,一遍DFS O(n+m)) · level[v]=DFS树深度,da[u][v]=min(level[v],每条back edge/v的level[v]),每个树节点的da[u][c].post-order自底向上算=v的子树最高能爬回哪层). 删剖(tree上→v):桥=da[u][v]=level[u] (子树连u够不着,此边=唯一通道). 删剖:非根u⇒某树孩子c有da[u][c]≥level[u](爬不到u的严格上方);

根⇒≥2个树孩子. ▲桥用u,删剖用u:判桥到u就算逃生(绕回u有别的路);判割点u本人被删,碰到u没访问,必须爬过u.

bipartite=无奇环⇒BFS无同层边;判定:BFS后偶层染红奇层染蓝,检查每条边两端异色,O(n+m). **O(n)环**:DFS只看前n条边——若n条内全是tree edge则≥n个点矛盾,必出back edge. 层鸽巢:dist(s,t)≥n/2-层数>n/2→若每层≥2点则点数>矛盾 · 某层只1点,删它断开,s,t.

T7套路:check-stuck点O(n):从(1,1)短阶片爬走,1→杀此行下移,0→杀此列右移,n步删1候选再O(n)验证; 连通分量编号:每分量standard标最小编号;子树聚合B[u]=min(A[u],孩子B)=post-order.

W8 Dijkstra-MST

算法	时间
Dijkstra:堆	O((n+m)log n)
Dijkstra F+数组	O(m+n log n/O(n²))
Prim 堆	O((n+m)log n)
Kruskal	O(m log m)+UF简单版O(n²)

Dijkstra过程:①D[s]=0,其余D[v]=∞,全放入min-PQ(按D) ②反复extract-min取出u(此到D[u]定型) ③对u每条出边(u,v)做relax:D[v]=min(D[v],D[u]+w(u,v)),变小就decrease-key,只删自己. **W5**:n次extractlog n+m次decrease-key log n→O((n+m)log n) ④PQ空止. 负权失效. 正确性:若extract时D[u]不是真最短,真最短路必经某未处理点u,但D[x]≥D[u]且后续边权≥0,绕路不可能更短(链:w(P)≥w(P')+w(x,y)≥D[x]+w(x,y)≥D[u])——矛盾. **SPT=SPMST**(Δ,1,1,1,1,1,1).

Prim过程:任选s起树T,PQ存有所有“T连出去的边”,每次取最短跨边,对端不在T就加入;重复直到T含n个点(每步由cut property合法).

W8 Dijkstra-MST(续)

Kruskal过程:①全部边按权升序排 ②从轻到重逐条看:两端在不同分量(union-find查)→加入合并;同分量·跳过(会成环) ③攒够n-1条止. 简单UF:array记每元素属哪个set(find O(1),union(u,v) O(n)这个懒;合并2m次+union n-1次→排序后O(n²)(DFS查环则O(mn)). ▲平局:等权→字典序/index tie-break.

cut=点集S,cutset=恰一端的S的边集;▲任何环与任何cutset相交偶数条边. 割性质(cut):把点集任意切成两半(S,V\S),跨割的边中最轻的e必在每个MST里. 证:设MST t不含e;T连通→T内必有另一跨割边;t+e成环,删掉T=t+e-仍仍是生成树且w(e)<w(f) · T更轻,矛盾.

环性质(cycle):任何环C上唯一最重的边必不在任何MST里. 证:设T含f,删掉T断成两半,C上必有另一条边e跨这两半且w(e)<w(f),T+f+e更轻的生成树,矛盾.

收尾套路:删掉轻的都∈MST→MST≤T;轻n-1条边→T=MST. **T8套路**:边权变换ℓ=ℓ-M(M>0)保序 · 最短路不变;ℓ=ℓ+1按边数惩罚→可能改变;取M=n+1可修老 · 点权最短路→拆点v→(vin,vout)核=点费,跑Dijkstra. ▲手跑Dijkstra/Kruskal:按字典序,逐步写表.

W9 贪心

范式:排序+迭代局部最优. 证明:exchange(逆序对相邻交换不变差)/stays ahead(归纳领先). 例:fractional背包可拆→按价值密度贪心最优(O(1)不可);interval sched=earliest finish:partitioning=depth个教室(depth=同时盖住某时刻的最大区间数=下界);加权完成=按T/v升序(边权差=-wtₐ+wₑtₑ). 反例要算出两个值:算得真X,最优得Y=X.

Huffman:字符集C频率f(c),高频→短码,前缀码:无码字是另一码字的前缀=字符全在叶;cost(T)=Σf(c)·depth_T(c):频最小两个先合,堆O(n+log d),最优性=两引理(最深两叶可换频率最小两个;合并后最优⇒合并前最优)+归纳. run-length encoding=连续重复字符记成计数. 贪心(k+1)染色:度< k的边,选该最小可用色(邻居最多占k色,k+1色足够=鸽巢),O(nk+m).

W10 分治I

范式:divide/conquer/combine;正确性=归纳. 解递:unrolling递层代(mergesort最慢都=cn:1×cn,2×cn/2,...,共log n层);递归树层比=a/b^d(deg);>1叶主导 O(n^d(log b, a_b))=1每层同log n,<1根主导 Θ(f₁)·1+...+r¹k=(r¹(k+1)-1)/(r-1),r<1时O(1);**Master**(T=aT(n-b)+f₁,f₁=O(n²):共n/b层,a=1→每层相加加O(n²(k+1));T(n)=T(n-1)+1 · O(n);n→O(n²);T(n-100n)+n→仍O(n²);a=1→指数O(n²aⁿ(n/b))T=2T(n-1)+1→O(2ⁿ);3T(n-1)+1→O(3ⁿ);2T(n-1)+n→O(n·2ⁿ)). ▲减常数(n-2)/除常数(n/b),前者层数线性后者log→别套Master! 逆序对:merge取右cnt+=|L|-i. 减半T(n/2)+O(1)=O(log n):二分/快速幂/缺失数/旋转峰值(只递归一边);二分要O(1)随机访问:array才行,linked list退化O(n);base:|S|<2.

比较	p范围	T(n)
d<k	任意	Θ(nᵈ)
d=k	p>-1	Θ(loglogᵈ(p+1)) (p=0:×log n)
	p=-1	Θ(m log m)
	p<-1	Θ(nᵈ)
d<k	p≥0	Θ(logᵈm)=Θ(f)
	p<0	Θ(nᵈ)

减治递推T(n)=aT(n-b)+f(n),f=O(n²):共n/b层,a=1→每层相加加O(n²(k+1));T(n)=T(n-1)+1 · O(n);n→O(n²);T(n-100n)+n→仍O(n²);a=1→指数O(n²aⁿ(n/b))T=2T(n-1)+1→O(2ⁿ);3T(n-1)+1→O(3ⁿ);2T(n-1)+n→O(n·2ⁿ)). ▲减常数(n-2)/除常数(n/b),前者层数线性后者log→别套Master!

逆序对:merge取右cnt+=|L|-i. 减半T(n/2)+O(1)=O(log n):二分/快速幂/缺失数/旋转峰值(只递归一边);二分要O(1)随机访问:array才行,linked list退化O(n);base:|S|<2.

mergesort:merge用O(n)比较+额外数组;正确性两个分开归纳(merge on iterations/sort on size). quicksort pivot:分<,>=三块(L/E/S),只递归两边;E[T(n)]=E[T(n)+T(n)]+O(n) · 期望O(n log n),最坏O(n²)(已序输入). ▲O(n²)记在哪看啥版本:三列表版divide发分O(n)+conquer拼回O(n);

原地版拼回交换=divide O(n),conquer O(1)——每层O(n)跑不掉,总时间相同,in-place只省空间;对照:mergesort功夫在combine(merge),quicksort功夫在divide(partition). 下界决策树:n!种排列-n!叶,高h二叉树≤2ⁿh叶→2ⁿh≥n!→h≥log₂(n!)≥(n/2)log(n/2)=Ω(n log n).

怪速推真题(W11 P3):new-sort:排前2/3→排后2/3→再排前2/3(T(n)=3T(2n/3)+O(1)=O(n²(1.5)3)) =O(2ⁿ(2.71));(很慢) · 正确性“三档论证”:值/位置各分小中大三段,当档证被推进正确段. 方块矩阵乘:8次乘=8T(n/2)+O(n²)=O(n³);Strassen 7次 · O(n²(2.81)) (减少4). **median-of-medians**(超预期)解:5个一组取各组中位数的中位数当pivot→rank∈[3n/10,7n/10]→T(7n/10)+T(n/5)+O(n),9/10<1→O(n);

△.3个一组:T(2n/3)+T(n/3)+O(n)每层都cn→O(n log n)不行.

△ 易错检查单(交卷前过一遍)

build-heap=O(n). SLL删尾O(n). heap左右无序. **SPT=AMST**:哈希=expected(写前摊还). DEFUNCT连续探/空槽停 O(h+s)别忘+s:增广要写维护·invariant写全所有变量,“always/必然”T/F警惕反例. Huffman根=频率自查+并排加套套乘·2n=O(n),谷底峰峰只递归一边,跨半自查. **单独双指针**:Kruskal简单UF O(n²). REVERSE-MST中途有环·amortized-worst-case·候选?答案要验证·桥?删点=黑盒算法报名字直接用,只记自己的归约.

③ 答题骨架(三段)

Describe: Divide怎么切/维护什么 → Conquer递归谁 → Combine怎么合 → Base case. 每句一个动作;每次“递归一边/丢一半”后必跟 because...

Correctness: 循环 → loop invariant三段;递归 → 归纳 Base/IH/Step.

Complexity: 递推+解(Master/展开) 或 计数论证. 三部分物理分段写!

复杂度反推法:题目指定技术必须用(写错技术=大扣分)!

O(n log n)+D&C → 改mergesort;O(nk+m) → 贪心扫描;

O(log n)查询 → 树/二分;O(1) → 无循环、维护变量;O(n) → 不能排序.

④ 万能句库

Describe开头: "We run X with a **minor modification**..." "We **maintain/keep** a ..." "We **reduce** the problem to ..." "We **augment** each node/entry with ..." "We **precompute** ... in O(n)" "....with a single post-order traversal".

Loop invariant三段:

"Invariant: before iteration i, [P → 写全每个变化的变量]; Initialization: holds trivially before the loop. Maintenance: if it holds before iteration i, then after the iteration [...] Termination: at the end the invariant gives [goal]."

递归正确性=归纳三段:

"Base: the smallest case returns correctly. IH: assume the recursive calls on smaller inputs are correct. Step: the combine step yields the correct result for size n."

Correctness零件: "Suppose, **for the sake of contradiction**, ... → a contradiction." "Notice that ... **never increases**, therefore ..." "We can **safely ignore/discard** it because ..." "By the **Pigeonhole Principle**..." "By the **BST property**..." "By **the cut/ cycle property**..."

Complexity: "Each node is visited once with O(1) work → O(n)." "dominated by ...: n ops × O(log k) = O(n log k)." "Zdeg=2m → O(n+m)." "two boundary root-to-leaf paths → one visit per reported item → O(n+s)." "O(1) **expected** given load factor ≤½." "The recurrence is T(n)=..., solving to ... by the Master Theorem / unrolling." "Each level does O(n) work over O(log n) levels → O(n log n)."

下界/反例: "We disprove it with a **counterexample**: on ..., the algorithm returns ..., but the optimum is ..." 比较排序 **Ω(n log n)**(决策树高≥log n) "Consider the array sorted in **descending order** ... last n/2 iterations each ≥n/2 → Ω(n²)."

黑盒引用: 数过的算法报名字+复杂度直接用:"compute an MST using Kruskal"."sort with merge sort O(n log n)" "run BFS O(n+m);**只证你的归约逻辑**;前提 (expected、α有界)必须写。

⑤ 贪心模板

Exchange argument(整段照写):

"Let O be an arbitrary optimal solution; if O ≠ greedy there is an inversion → an adjacent pair ordered differently. Swapping them does not increase the cost because [...]. Each swap removes one inversion, so after finitely many swaps O becomes the greedy solution with no greater cost; hence greedy is optimal."

Stays ahead: 归纳 f(g_r)≤f(o_r) → 贪心永不先卡死 (interval scheduling=earliest finish).

⑥ 分治模板

★分治**Describe**万能模板(填空直接套):

"We solve this with divide and conquer by **modifying merge sort. Divide:** split the input into two halves. **Conquer:** recursively solve each half → each call returns [子问题答案,如:the count within that half / the sorted half / the best within the half]. **Combine:** [合并动作,如:count the cross pairs with a SEPARATE two-pointer pass over the two sorted halves / take the maximum of (left, right, crossing) / merge the two sorted halves]. **Base case:** an input of size ≤1 returns [trivial/答案]. The recurrence is T(n)=2T(n/2)+O(n), which solves to **O(n log n)** by the Master Theorem (a=2, b=2, f(n)=O(n))."

只递归一边版:把Conquer改成 "we recurse into ONLY the half that can contain the answer, discarding the other half **because [理由]**";递推T(n)=T(n/2)+O(1)=O(log n).

减半(只递归一边):

"Compare the middle ...; the half that cannot contain the answer is discarded; one recursive call on half the input: T(n)=T(n/2)+O(1)=O(log n)."

两表第k小 O(log(m+n)):比A[k/2-1] vs B[k/2-1];

"the first k/2 elements of the smaller list are all among the smallest k-1 of the union, so none can be the k-th → discard them, k←k/2."

base:表空/k=1,k/2 剩余;挪下标O(1). 中位数=(第+m+1)/2小.

数对子(改mergesort):

"total = within left + within right + cross; the halves are handled by recursion; the cross count is a SEPARATE two-pointer pass over the two sorted halves (the merge step only sorts). 2T(n/2)+O(n)=O(n log n)."

逆序对:merge取右时 **cnt += |L|-i**(唯一—新行). 电线交叉对(a_i,p_i):交叉⇐a<a_i且p_i>p_i → 按a排序后数p的逆序对;双链表输入照样O(n log n)(找中点O(n)被吸收);已按a排好序!O(n log n)(数逆序是瓶颈). **窗口机制**译:LR已各自有序;遍历R的每个b,在L上维护两指针lo=首个≤窗左沿,hi=首个>

⑦ 分治模板(续)

窗右沿,该b贡献**cnt+=hi-lo**;指针全程**不重置**(重置→O(n²)). **|a-b|≤d:**窗=[b-d,b+d],b升序→窗右滑,lo/hi单调右移,各走≤|L|步 → cross=O(n). 和**∈[lo_o,hi_o]**:对每个b窗=[lo_o-b,hi_o-b],**b越大窗越靠左**→指针单调**左移**(或b倒序),同O(n). **a>2b:**数L中>

2b的个数,阈值随b右移;必须单独指针,并入merge负数会错.

Kadane O(n):s=max(A[i],s+A[i]),best=max(best,s);**invariant**必须同时描述**目标和best**.

★最大子数组D&C版(题目指定D&C必用此版):切两半,答案=**max**(左半内,右半内,跨中点)三类取**MAX**——不是相加,和数对子不同!跨中点=从mid往左的最大后缀和+往右的最大前缀和(各一遍线性扫O(n)). 2T(n/2)+O(n)=O(n log n).

最近点对(平面**n**点):按x切两半,δ=min(左半最近,右半最近);再查中线、x距中线<δ的窄带;带内按y排序,每点只和附近常数个(≤7)比. 2T(n/2)+O(n)=O(n log n).

majority:>n/2;两半各拿候选+**O(n)**验证=O(n log n)(引理:全局多数必是某半多数,反证两半都≤½加起来≤n/3);或配对消除O(n). >n/3:至多2个候选;2候选位+2计数器扫一遍;命中+1有空位上位/否则三个互不相同 → 双-1(扔3个不同的,>n/3者最多损1,扔不完);结尾**O(n)**逐个验证,可能**O(1/2)**个答案;

>n/k → k-1个候选.

快速幂x^n:x^n=n=(x^(n/2))^n奇再×x;T(n)=T(n/2)+O(1)=O(log n);正确性归纳:base n=0 → 1;step由x^n=n=(x^(|n/2|))^(-x).

median-of-medians选pivot(了解):5个一组取组中位数的中位数 → rank∈[3n/10,7n/10] → T(7n/10)+T(n/5)+O(n), **7/10+1/5<1 → O(n)**;△3个一组T(2n/3)+T(n/3)+O(n):2/3+1/3=1每层cn → O(n log n)失败.

W11 问题 II

问题	时间
mergesort/逆序对/数对子 maxima set	O(n log n)
最大子数组 D&C / Kadane	O(n log n) O(n)
quickselect	exp O(n) ,worst O(n²)
两表第k小	O(log(m+n))
majority >n/2,>n/3	O(n)
Karatsuba/Strassen	O(n¹.585)O(n².81)

最大子数组:跨=最大后缀+前缀;Kadane s=max(A[i],s+A[i]). **数对子窗口:**cross单独双指针(merge只排序);对R每个b,L上lo=首个≥窗左沿/hi=首个>右沿,**cnt+=hi-lo**,指针**不重置**;|a-b|≤d → 窗=[b-d,b+d]随b右滑,和∈[lo_o,hi_o] → 窗[lo_o-b,hi_o-b]随b左滑;

a>2b → 阈值右移. **电线交叉:**按a排序数p逆序. 第k小:比A[k/2] vs B[k/2]丢k/2. **最近点**对:按x切两半,δ=min(左半,右半);跨中线只查x距<δ的窄带;带内按y排每点只比附近常数个 → 2T(n/2)+O(n)=O(n log n). **majority>n/2:**D&C各半拿候选+O(n)验证=O(n log n)(引理:全局多数必是某半多数);

或配对消除O(n). >n/3:至多**2**个候选(3个就总数>n):2候选位+2计数器,命中+1空位上位/否则三个互不相同 → 双计数-1(扔3个不同的,>n/3者最多损1,扔不完);结尾**必须 O(n)**验证,候选**★答案**;>n/k → k-1候选. **merge-k:**堆O(mk log k);或两半D&C合并,log k层每层O(mk),同O(mk log k).

maxima set/Pareto:g支配p⇐q.x≥p.x且q.y≥p.y;p是 maximum⇐无人支配它. 按x排(同x按y)O(n log n);D&C:右半x全更大 → 右半**maxima**全保留,左半只留y>右半最大y者,2T(n/2)+O(n)=O(n log n)(或排好后右 → 左扫O(n)). 整数乘法:×=x₁2^n/2+×₀拆半,直接4T(n/2)+O(n)=O(n²)(x²*k=移位O(n));

Karatsuba:z₁=(x₁+x₀)(y₁+y₀)-z₂-z₀省一次乘 → 3T(n/2)+O(n)=**O(n¹.585)**;Strassen 7乘O(n¹.281)(思想:减少子问题数a).

⑧ 树·节点存值模板

看孩子 → post-order(先递归后算):

"We compute X with a single post-order traversal: for each node u first recursively compute X of its children, then set u's X to [combine]; an empty child contributes [base]. Each node visited once with O(1) work → O(n)."

size=L+R+1(base 0) · height=max(L,R)+1(base -1,叶0) · balance=|hL-hR| · 子树sum=L+R+u.key.

看父亲 → pre-order(先算后传):

"...pre-order traversal passing the parent's value down: set u's X=g(parent) before recursing into children; the root starts with [ini]."

depth=parent.depth+1(根0) · 到根路径和=parent.val+u.key.

直径/最长路径:

"post-order returns each node's height; at each u update a global maximum with hL+hR+2; after the traversal the maximum is the diameter. O(n)."

问长度不用记端点.

Successor三套: **pre:**有左 → 左孩;无左有右 → 右孩;叶 → 上溯到「u在其左子树且有右孩」的祖先v→v的右孩. **in:**有右 → 右子树最左;无右 → 上溯第一次「左拐上来」的祖先. **post:**根 → 无;u是右孩 → 父亲;u是左孩 → 父亲有右孩? 父子子树post-order第一个(能左则左否则右,到叶);父亲.

⑨ Range search + 增广BST

"To report all keys in [k1,k2] we start at the root and recurse. At a node u: if u.key<k1, every key in u's left subtree is also <k1, so we recurse only into the right subtree; if u.key>k2 we recurse only left; otherwise k1≤u.key≤k2, so we report u.key and recurse into both subtrees."

Correctness(induction on subtree):base空树 ✓ ;IH两递归正确;step 3 case—由BST性质剪掉的半边不含range内key(无漏),且只报范围内. **Time:**递归沿k1,k2两条根-叶边界路径(O(h)) + 每个报告点访问一次 → **O(h+s)**. 只数个 数:report count++ ,同O(h+s)("直接扫"). remove-all(k1,k2)同框架.

增广**BST**(动态+反复查才用;要写维护)

⑩ Range search + 增广BST(续)

每点存子树**size**=L+R+1,ins/del沿路O(1)更新. **select(k)**:t=size(left)+1(=u的排名,≠子树大小);k=t → u; k<t → 左找第k; k>t → 右找第k-t;O(h). 一次性查询用in-order数到第k(O(k)),临时标size=O(n)不划算. **rank(x)=#<x:**下降>>u.key → 加size(left)+1走右;否则左;O(h). **区间个数= ranks(k2)-rank<(k1)**(整数:rank<(k2+1)-rank<(k1)). 区间和:size换子树sum. 信号:"supports insert/delete+query, each O(log n)". AVL存height也是增广.

★**Range&missing-min**结构:AVL(+size)+三变量min/max/miss;

get-range/get-missing-min返回变量**O(1)**;insert(i)更新miss四case:①树空 → miss=None ②i<min:贴边(i=min-1)不变;隔开 → **miss=i+1** ③i>max:贴边不变;隔开 → miss=(原miss存在?原miss:max+1) ④内部:i**miss**不变;只有i=miss才重置.O(log n)下降找下一缺失 → 判据 [min,x]完整⇐**ranks(x)-x=**min+1,每步用左子树size判缺口在左/右; size=max-min+1⇐miss=None. △**select MCQ:k>L+1**常移项写成**L<k-1(=进右找k-L-1)**.

同型扩展·每个的Describe(英文照写) **区间计数count(k1,k2):**

"Augment each node with its subtree size. rank(x) (= number of keys <x) is computed by one root-to-leaf descent: whenever we go right at u we add size(u.left)+1. Then count(k1,k2) = ranks(k2) - rank<(k1). O(log n)."

第k大:

"The k-th largest is the (n-k+1)-th smallest, so we return select(n-k+1) using the same size-augmented descent."

动态中位数/百分位:

"We maintain a size-augmented AVL tree under insertions and deletions; the median is select((n/2)) (the p-th percentile is select([p·n])), one descent, O(log n) per operation."

区间求和sum(k1,k2):

"We augment each node with the sum of keys in its subtree (sum = left.sum + right.sum + u.key, maintained in O(1) per node on the search path). prefixSum(x) descends from the root, adding sum(u.left)+u.key whenever it goes right; the answer is prefixSum(k2) - prefixSum(k1-1). O(log n)."

区间min/max:

"No augmentation needed: descend as in range search; the smallest key in [k1,k2] is the leftmost node reached with key ≥ k1 (and ≤ k2). O(n)."

完整性/缺失判定:

"For distinct integers, [min,x] is complete iff ranks(x) = x - min + 1 (expected count = actual count), one descent guided by left-subtree sizes finds the smallest violation = smallest missing integer."

增广合法性+维护句(必写):

"Each augmented field can be recomputed from its two children in O(1), so insert/delete update it along the search path only, keeping O(log n) per operation."

★**线段查询(几条线段含x):**含x⇐left<x且right>x ⇐ **count=#{left<x}-#{right<x}**;预处理:lefts、rights各排成有序数组O(n log n);查询=两次二分O(log n).

★**动态矩阵:**上三角isj → **columns[j][i]**;下三角j → **rows[i][j]**(columns[j]长j,rows[i]长i-1);get/set O(1); **increase-size:**新索引数组+复制n个指针+新建一行一列清零 → **O(n)**(不复制n²数据);空间O(n²).

⑪ 图/MST 套句

O(n)环(连通无向):

"Run DFS with a minor modification: when scanning u's neighbours, if v is visited and v≠parent(u) → cycle v,u,parent(u),...,v. Until found we have examined only tree edges, at most n-1 → O(n)."

BFS反证(**dist(s,t)>n/2** → 割点):BFS自s,t∈L_k,k>n/2;若每层≥2点则|L_i|≥2(k-1)>n-2矛盾 → 某层单点;

"Suppose, for the sake of contradiction, an s-t path avoids u; it has an edge (x,y), x∈L_a, y∈L_b, a<b → but BFS would have placed y in L_a+1). Contradiction."

点权最短路归约(T8P5):

"Split each v into vin → vout joined by an edge of weight = v's cost; every edge (u,v) becomes uout → vin; then run Dijkstra. O((n+m)log n)."

哈希万能开头:

"Keep a hash table with 2n entries using linear probing: since the load factor is ≤½, each operation takes O(1) expected time."

用例:众数exp O(n)(边扫边维护best)·生日表2·365, >365人366内必撞 → O(1)-k-gram多项式hash → O(n)·查重 set-two-sum:partner=t-x先查后插.

堆套句:第k小=size-k max-heap O(n log k)·"safely ignore A[i]: the queue already holds k values smaller." 合并k表 min-heap O(mk log k)·"the minimum never goes down ⇐ output sorted." insertion sort=O(n+1);每swap消1逆序,Σk_i=l_i-l_n=l(望远镜).

W13 原理·考点

鸽巢原理:把m>N个物品放进N个盒子,必有盒子≥2个;推广:必有盒子≥|mN|个. 用法:①固定哈希h、取nN+1个key必有n个同槽(坏数据集必存在) ②BFS层:层数>n/2且每层≥2点则>n矛盾 ③n+1个数取自n个值必重复.

exchange argument:任取最优解O;若O≠贪心,存在相邻一对顺序与贪心不同(逆序);交换这对代价不增(逐项算差);有限次交换后O变成贪心解且不更差 → 贪心最优.

复杂度反推:n log n → 排序/D&C;nk+m → 贪心扫;log n → 树/二分;O(1) → 维护变量;O(n) → 禁排序.

高频:数对子·增广DS·stack应用·鸽巢·贪心反例·手跑算法·merge-k·三段答题(describe/prove/analyse).